Experiment Number: 05

Title: Write a program to implement Link State / Distance Vector routing protocol to find a suitable path for transmission.

Problem Statement: Find suitable path for transmission using Link State and Distance Vector routing protocol concepts.

Objectives:

1. To understand the basic concept of routing protocols.
2. To implement Distance Vector routing protocol.
3. To implement Link State routing protocol.

Outcomes: Students will be able to,

1. Understand routing protocol behavior.
2. Implement Distance Vector routing protocol.
3. Implement Link State routing protocol.

Theory:

1) Distance Vector Routing Protocol: In Distance Vector routing, each router maintains a table of minimum cost (distance) to every destination and shares it with neighbors periodically. Routers update their table using Bellman-Ford style updates.

2) Link State Routing Protocol: In Link State routing, every router creates a map of the full network topology and computes shortest paths using Dijkstra's algorithm. OSPF is a practical example of a link-state protocol.

Code and Output:

Distance Vector Routing (C++):

```
#include <iostream>
#include <vector>
#include <iomanip>
using namespace std;
```

```
const int INF = 100000000;

int main() {
    // Adjacency matrix: cost between routers
    vector<vector<int>> cost = {
        {0, 2, 5, INF},
        {2, 0, 1, 4},
        {5, 1, 0, 3},
        {INF, 4, 3, 0}
    };

    int n = static_cast<int>(cost.size());
    vector<vector<int>> dist = cost;

    // Bellman-Ford style relaxation for all pairs
    for (int k = 0; k < n - 1; k++) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                for (int v = 0; v < n; v++) {
                    if (dist[i][v] < INF && dist[v][j
                        dist[i][j] = min(dist[i][j],
                    }
                }
            }
        }
    }

    cout << "Distance Vector Routing Table (Minimum C
    cout << "    ";
    for (int j = 0; j < n; j++) cout << "R" << j << "
    cout << "\n";

    for (int i = 0; i < n; i++) {
        cout << "R" << i << "  ";
        for (int j = 0; j < n; j++) {
            if (dist[i][j] >= INF) cout << setw(3) <<
            else cout << setw(3) << dist[i][j];
            cout << " ";
        }
        cout << "\n";
    }

    return 0;
}
```

Output:

```
Distance Vector Routing Table (Minimum Cost):
    R0  R1  R2  R3
```

```
R0    0   2   3   6
R1    2   0   1   4
R2    3   1   0   3
R3    6   4   3   0
```

Link State Routing (C++ using Dijkstra):

```cpp
#include <iostream>
#include <vector>
#include <queue>
#include <utility>
using namespace std;

const int INF = 100000000;

int main() {
    // Graph as adjacency matrix (same topology)
    vector<vector<int>> graph = {
        {0, 2, 5, 0},
        {2, 0, 1, 4},
        {5, 1, 0, 3},
        {0, 4, 3, 0}
    };

    int n = static_cast<int>(graph.size());
    int source = 0;
    vector<int> dist(n, INF);
    vector<int> parent(n, -1);

    priority_queue<pair<int, int>, vector<pair<int, i
    dist[source] = 0;
    pq.push({0, source});

    while (!pq.empty()) {
        auto top = pq.top();
        pq.pop();
        int d = top.first;
        int u = top.second;

        if (d != dist[u]) continue;

        for (int v = 0; v < n; v++) {
            if (graph[u][v] > 0 && dist[u] + graph[u]
                dist[v] = dist[u] + graph[u][v];
                parent[v] = u;
                pq.push({dist[v], v});
            }
        }
    }
```

```
    cout << "Link State Routing (Dijkstra) from R" <<
    for (int i = 0; i < n; i++) {
        cout << "Destination R" << i << " -> Cost: "
    }

    return 0;
}
```

Output:

```
Link State Routing (Dijkstra) from R0:
Destination R0 -> Cost: 0
Destination R1 -> Cost: 2
Destination R2 -> Cost: 3
Destination R3 -> Cost: 6
```

Conclusion: Thus, we have successfully implemented and understood Distance Vector and Link State routing protocol algorithms for shortest-path transmission.

Experiment Number: 06

Title: Use Packet Tracer tool for configuration of 3 router network using one of the following protocols: RIP / OSPF / BGP.

Problem Statement: Configure and verify a 3-router network using routing protocol configuration in Cisco Packet Tracer.

Objectives:

1. To understand routing protocols RIP, OSPF, and BGP.
2. To configure routers and verify end-to-end connectivity.

Outcomes: Students will be able to,

1. Understand practical router configuration workflow.
2. Configure multi-router network using RIP/OSPF/BGP in Packet Tracer.

Theory:

1) RIP (Routing Information Protocol): Distance-vector protocol using hop-count metric. Maximum hop count is 15.

2) OSPF (Open Shortest Path First): Link-state protocol using SPF algorithm and cost metric.

3) BGP (Border Gateway Protocol): Path-vector protocol used between autonomous systems in large networks.

Procedure (RIP Example with 3 Routers):

1. Place 3 routers, 3 switches, and 6 PCs in Packet Tracer.
2. Configure IP addresses on PCs and router interfaces.
3. Enable RIP and advertise connected networks.
4. Verify connectivity using `ping`, `show ip route`, and `show ip protocols`.

RIP Configuration Commands:

```
Router0(config)# router rip
Router0(config-router)# network 192.168.10.0
Router0(config-router)# network 10.0.0.0
```

```
Router1(config)# router rip
Router1(config-router)# network 192.168.20.0
Router1(config-router)# network 10.0.0.0
Router1(config-router)# network 11.0.0.0

Router2(config)# router rip
Router2(config-router)# network 192.168.30.0
Router2(config-router)# network 11.0.0.0
```

Verification Commands:

```
PC0> ping 192.168.30.2
Router0# show ip route
Router0# show ip protocols
```

Output (Sample):

```
PC0> ping 192.168.30.2
Reply from 192.168.30.2: bytes=32 time<1ms TTL=127
Reply from 192.168.30.2: bytes=32 time<1ms TTL=127
Reply from 192.168.30.2: bytes=32 time<1ms TTL=127
Reply from 192.168.30.2: bytes=32 time<1ms TTL=127

Success rate is 100 percent (4/4)
```

Conclusion: Thus, we have successfully configured and verified a 3-router network in Cisco Packet Tracer using routing protocol commands.

Experiment Number: 07

Title: Write a program using TCP socket for wired network for the following: (a) Say hello to each other, (b) File transfer, (c) Calculator, OR write a program using UDP sockets to enable file transfer between two machines.

Objectives:

1. Set up TCP connection between two nodes.
2. Implement hello message exchange, file transfer, and calculator operation using TCP.
3. Set up UDP transfer between two nodes for different file types.

Outcomes: Students will be able to,

1. Understand TCP and UDP socket programming.
2. Implement client-server communication in TCP.
3. Implement file transfer using UDP datagrams.

Theory:

1) What is a Socket? A socket is an endpoint of communication between two processes over a network.

2) Berkeley Socket Primitives: Key primitives include `socket()`, `bind()`, `listen()`, `accept()`, `connect()`, `send()/recv()`, and `close()`.

3) TCP Socket Programming: Connection-oriented, reliable byte stream with acknowledgment and retransmission.

4) UDP Socket Programming: Connectionless communication using datagrams with lower overhead.

Code and Output:

A) TCP Server (`server.cpp`)

```
#include <iostream>
#include <unistd.h>
#include <string.h>
#include <arpa/inet.h>
#include <fstream>
```

```
using namespace std;

int main() {
    int server_fd, new_socket;
    sockaddr_in address{};
    socklen_t addrlen = sizeof(address);
    char buffer[1024] = {0};

    server_fd = socket(AF_INET, SOCK_STREAM, 0);
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(5000);

    bind(server_fd, (sockaddr *)&address, sizeof(addr
    listen(server_fd, 3);

    cout << "Server waiting for connection...\n";
    new_socket = accept(server_fd, (sockaddr *)&addre
    cout << "Client connected!\n";

    while (true) {
        memset(buffer, 0, sizeof(buffer));
        int r = read(new_socket, buffer, sizeof(buffe
        if (r <= 0) break;

        string data(buffer);

        if (data == "HELLO") {
            send(new_socket, "Hello from Server!", 18
        } else if (data.rfind("FILE", 0) == 0) {
            ofstream file("received_file.txt", ios::b
            while (true) {
                memset(buffer, 0, sizeof(buffer));
                int bytes = read(new_socket, buffer,
                if (bytes <= 0 || strncmp(buffer, "EO
                file.write(buffer, bytes);
            }
            file.close();
            send(new_socket, "File received successfu
        } else if (data.rfind("CALC", 0) == 0) {
            string expr = data.substr(5);
            int a, b;
            char op;
            sscanf(expr.c_str(), "%d%c%d", &a, &op, &

            int result = 0;
            switch (op) {
                case '+': result = a + b; break;
                case '-': result = a - b; break;
                case '*': result = a * b; break;
                case '/': result = (b != 0) ? a / b :
                default: result = 0;
```

```
            }

            string out = to_string(result);
            send(new_socket, out.c_str(), out.length(
        } else if (data == "EXIT") {
            break;
        }
    }

    close(new_socket);
    close(server_fd);
    return 0;
}
```

A) TCP Client (`client.cpp`)

```
#include <iostream>
#include <unistd.h>
#include <arpa/inet.h>
#include <string.h>
#include <fstream>
using namespace std;

int main() {
    int sock = socket(AF_INET, SOCK_STREAM, 0);
    sockaddr_in serv_addr{};
    char buffer[1024] = {0};

    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(5000);
    inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_ad
    connect(sock, (sockaddr *)&serv_addr, sizeof(serv

    while (true) {
        int choice;
        cout << "\n1. Say Hello\n2. Send File\n3. Cal
        cout << "Enter choice: ";
        cin >> choice;
        cin.ignore();

        memset(buffer, 0, sizeof(buffer));

        if (choice == 1) {
            send(sock, "HELLO", 5, 0);
            read(sock, buffer, sizeof(buffer));
            cout << "Server: " << buffer << "\n";
        } else if (choice == 2) {
            ifstream file("send.txt", ios::binary);
            if (!file) {
```

```
                cout << "File not found\n";
                continue;
            }
            send(sock, "FILE", 4, 0);
            while (!file.eof()) {
                file.read(buffer, sizeof(buffer));
                send(sock, buffer, file.gcount(), 0);
            }
            send(sock, "EOF", 3, 0);
            memset(buffer, 0, sizeof(buffer));
            read(sock, buffer, sizeof(buffer));
            cout << buffer << "\n";
            file.close();
        } else if (choice == 3) {
            string expr;
            cout << "Enter expression (e.g., 10+5): "
            cin >> expr;
            string msg = "CALC " + expr;
            send(sock, msg.c_str(), msg.length(), 0);
            read(sock, buffer, sizeof(buffer));
            cout << "Result: " << buffer << "\n";
        } else if (choice == 4) {
            send(sock, "EXIT", 4, 0);
            break;
        }
    }

    close(sock);
    return 0;
}
```

B) UDP Server (`udp_server.cpp`)

```
#include <iostream>
#include <arpa/inet.h>
#include <unistd.h>
#include <fstream>
#include <cstring>
using namespace std;

#define PORT 5001
#define BUFFER_SIZE 1024

int main() {
    int sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    char buffer[BUFFER_SIZE];
    sockaddr_in server_addr{}, client_addr{};
    socklen_t addr_len = sizeof(client_addr);
```

```
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);
    server_addr.sin_addr.s_addr = INADDR_ANY;
    bind(sockfd, (sockaddr *)&server_addr, sizeof(ser

    cout << "UDP Server is running and waiting...\n";

    for (int i = 0; i < 4; i++) {
        memset(buffer, 0, BUFFER_SIZE);
        recvfrom(sockfd, buffer, BUFFER_SIZE, 0, (soc

        string filename = "received_" + string(buffer
        ofstream file(filename, ios::binary);
        cout << "Receiving file: " << filename << "\n

        while (true) {
            memset(buffer, 0, BUFFER_SIZE);
            int bytes = recvfrom(sockfd, buffer, BUFF
            if (bytes <= 0 || strncmp(buffer, "EOF",
            file.write(buffer, bytes);
        }

        file.close();
        cout << "File received successfully: " << fil
    }

    cout << "All 4 files received successfully.\n";
    close(sockfd);
    return 0;
}
```

B) UDP Client (`udp_client.cpp`)

```
#include <iostream>
#include <arpa/inet.h>
#include <unistd.h>
#include <fstream>
#include <cstring>
using namespace std;

#define PORT 5001
#define BUFFER_SIZE 1024

int main() {
    int sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    char buffer[BUFFER_SIZE];
    sockaddr_in server_addr{};

    server_addr.sin_family = AF_INET;
```

```
    server_addr.sin_port = htons(PORT);
    inet_pton(AF_INET, "127.0.0.1", &server_addr.sin_

    string files[4];
    cout << "Enter Script file name: "; cin >> files[
    cout << "Enter Text file name: "; cin >> files[1]
    cout << "Enter Audio file name: "; cin >> files[2
    cout << "Enter Video file name: "; cin >> files[3

    for (int i = 0; i < 4; i++) {
        ifstream file(files[i], ios::binary);
        if (!file) {
            cout << "File not found: " << files[i] <<
            continue;
        }

        sendto(sockfd, files[i].c_str(), files[i].len
        cout << "Sending file: " << files[i] << "\n";

        while (!file.eof()) {
            file.read(buffer, BUFFER_SIZE);
            sendto(sockfd, buffer, file.gcount(), 0,
        }

        sendto(sockfd, "EOF", 3, 0, (sockaddr *)&serv
        file.close();
        cout << "File sent successfully: " << files[i
    }

    cout << "All 4 files sent successfully.\n";
    close(sockfd);
    return 0;
}
```

Compilation and Execution (Linux/Unix):

```
g++ server.cpp -o server
g++ client.cpp -o client
./server
./client

g++ udp_server.cpp -o udp_server
g++ udp_client.cpp -o udp_client
./udp_server
./udp_client
```

Output (Sample):

```
Server waiting for connection...
Client connected!

Client Menu:
1. Say Hello
2. Send File
3. Calculator
4. Exit

Enter choice: 1
Server: Hello from Server!

Enter choice: 3
Enter expression (e.g., 10+5): 10+5
Result: 15

Enter choice: 2
File received successfully

UDP Server is running and waiting...
Receiving file: received_script.cpp
File received successfully: received_script.cpp
...
All 4 files received successfully.
```

Conclusion: Thus, we have successfully implemented socket programming using both TCP and UDP for communication and file transfer.

Experiment Number: 08

Title: Write a program for DNS lookup. Given an IP address as input, it should return URL and vice-versa.

Problem Statement: Map hostname to IP address (forward lookup) and IP address to hostname (reverse lookup) using DNS.

Objectives:

1. To get hostname and IP address information.
2. To map hostname with IP address and vice-versa.

Outcomes: Students will be able to,

1. Understand the need of DNS.
2. Implement a DNS lookup utility program.

Theory:

1) What is DNS? DNS (Domain Name System) translates human-readable domain names into IP addresses and also supports reverse lookup through PTR records.

2) How TCP/IP uses DNS: A DNS client sends query to DNS server to resolve name-to-address mapping before establishing network communication.

Code and Output:

DNS Lookup Program (C++ / POSIX):

```
#include <iostream>
#include <arpa/inet.h>
#include <netdb.h>
#include <cstring>
using namespace std;

void forwardLookup(const string &host) {
    hostent *he = gethostbyname(host.c_str());
    if (he == nullptr) {
        herror("gethostbyname");
        return;
    }
```

```
    in_addr **addr_list = (in_addr **)he->h_addr_list
    cout << "IP address(es) of " << host << ":\n";
    for (int i = 0; addr_list[i] != nullptr; i++) {
        cout << inet_ntoa(*addr_list[i]) << "\n";
    }
}

void reverseLookup(const string &ip) {
    sockaddr_in sa{};
    sa.sin_family = AF_INET;

    if (inet_pton(AF_INET, ip.c_str(), &sa.sin_addr)
        cout << "Invalid IP address format\n";
        return;
    }

    hostent *he = gethostbyaddr(&sa.sin_addr, sizeof(
    if (he == nullptr) {
        herror("gethostbyaddr");
        return;
    }

    cout << "Domain name of " << ip << ": " << he->h_
}

int main() {
    int choice;
    string input;

    cout << "DNS Lookup Program\n";
    cout << "1. Forward Lookup (Domain -> IP)\n";
    cout << "2. Reverse Lookup (IP -> Domain)\n";
    cout << "Enter your choice: ";
    cin >> choice;

    if (choice == 1) {
        cout << "Enter domain name: ";
        cin >> input;
        forwardLookup(input);
    } else if (choice == 2) {
        cout << "Enter IP address: ";
        cin >> input;
        reverseLookup(input);
    } else {
        cout << "Invalid choice!\n";
    }

    return 0;
}
```

Compilation and Execution (Linux/Unix):

```
g++ dns_lookup.cpp -o dns_lookup
./dns_lookup
```

Output (Sample):

```
DNS Lookup Program
1. Forward Lookup (Domain -> IP)
2. Reverse Lookup (IP -> Domain)
Enter your choice: 1
Enter domain name: www.google.com
IP address(es) of www.google.com:
142.250.192.196

DNS Lookup Program
1. Forward Lookup (Domain -> IP)
2. Reverse Lookup (IP -> Domain)
Enter your choice: 2
Enter IP address: 142.250.192.196
Domain name of 142.250.192.196: del03s06-in-f4.1e100.
```

Conclusion: Thus, we have successfully implemented DNS forward and reverse lookup program.